

Ch04. word2vec 속도 개선



■ 주요 내용

- word2vec 개선 (1) – Embedding 계층
- word2vec 개선 (2) – 네거티브 샘플링
- 개선판 word2vec 학습
- word2vec 남은 주제

word2vec 속도 개선

■ CBOW 모델 구현의 문제점

- 단순한 2층 신경망
- 말뭉치에 포함된 어휘 수가 많아지면 계산량도 커짐
- 어휘수가 많아지면 계산 시간이 너무 오래 걸림

■ word2vec 속도 개선

- Embedding 계층 도입
- 네거티브 샘플링 도입
 - 새로운 손실 함수
- PTB 데이터셋을 가지고 학습을 수행

4.1 word2vec 개선 (1)

그림 4-1 앞장에서 구현한 CBOW 모델

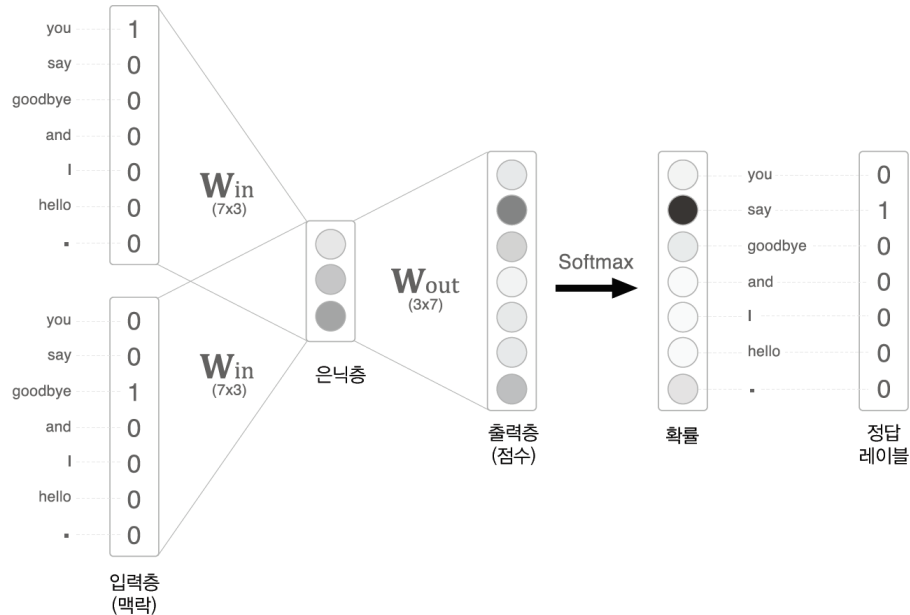
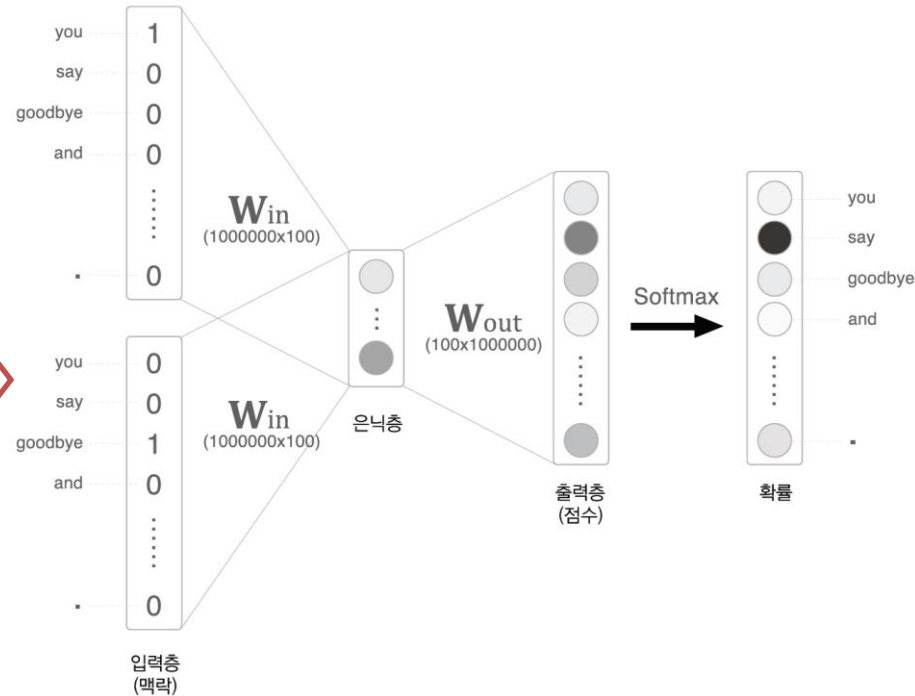


그림 4-2 어휘가 100만 개일 때를 가정한 CBOW 모델



■ 거대한 말뭉치를 다루게 될때의 문제점

■ 입력층의 원핫 표현과 가중치 행렬 W_{in} 의 곱 계산

- 어휘수가 많아지면 원핫 표현의 벡터 크기도 커짐, 원핫 벡터와 W_{in} 을 곱해야 함

■ 은닉층과 가중치 행렬 W_{out} 의 곱 및 Softmax 계층의 계산

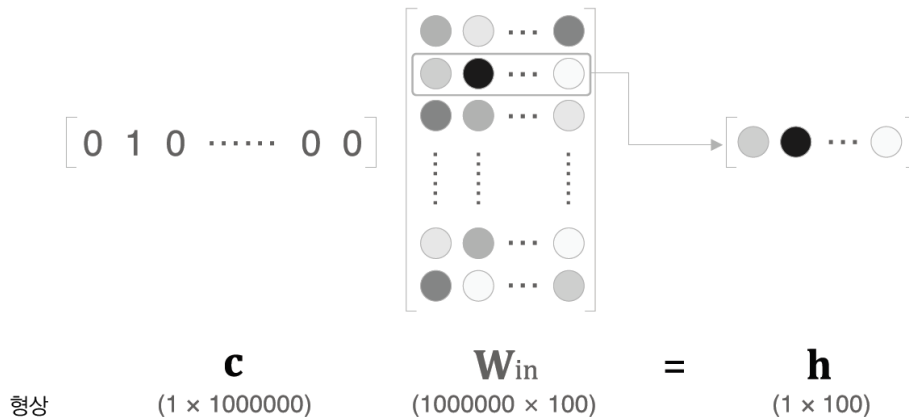
- 은닉층과 W_{out} 의 곱만 해도 계산량이 많고, Softmax 계층에서도 계산량이 증가함

4.1.1 Embedding 계층

■ MatMul 계층의 행렬 곱

- 어휘 수가 100만 개이고, 은닉층 뉴런이 100개인 경우
 - 단어의 원핫 표현 100만 차원과 가중치 행렬을 곱해야 함

그림 4-3 맥락(원핫 표현)과 MatMul 계층의 가중치를 곱한다.



■ 수행하는 일 – 행렬의 특정 행을 추출

-> 원핫 표현으로의 변환과 MatMul 계층의 행렬 곱 계산이 필요 없음

■ Embedding 계층 생성

- 가중치 매개변수로부터 "단어 ID에 해당하는 행(벡터)"을 추출하는 계층
- Embedding 계층에 단어 임베딩(분산 표현)을 저장하는 것

4.1.2 Embedding 계층 구현

- 행렬에서 특정 행을 추출 방법
 - 가중치 W가 2차원 배열일 때,
가중치로부터 특정 행을 추출할시
 - W[2]나 W[5] 처럼 원하는 행을 명시

```
>>> import numpy as np
>>> W = np.arange(21).reshape(7,3)
>>> W
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14],
       [15, 16, 17],
       [18, 19, 20]])
>>> W[2]
array([ 6,  7,  8])
>>> W[5]
array([15, 16, 17])
```

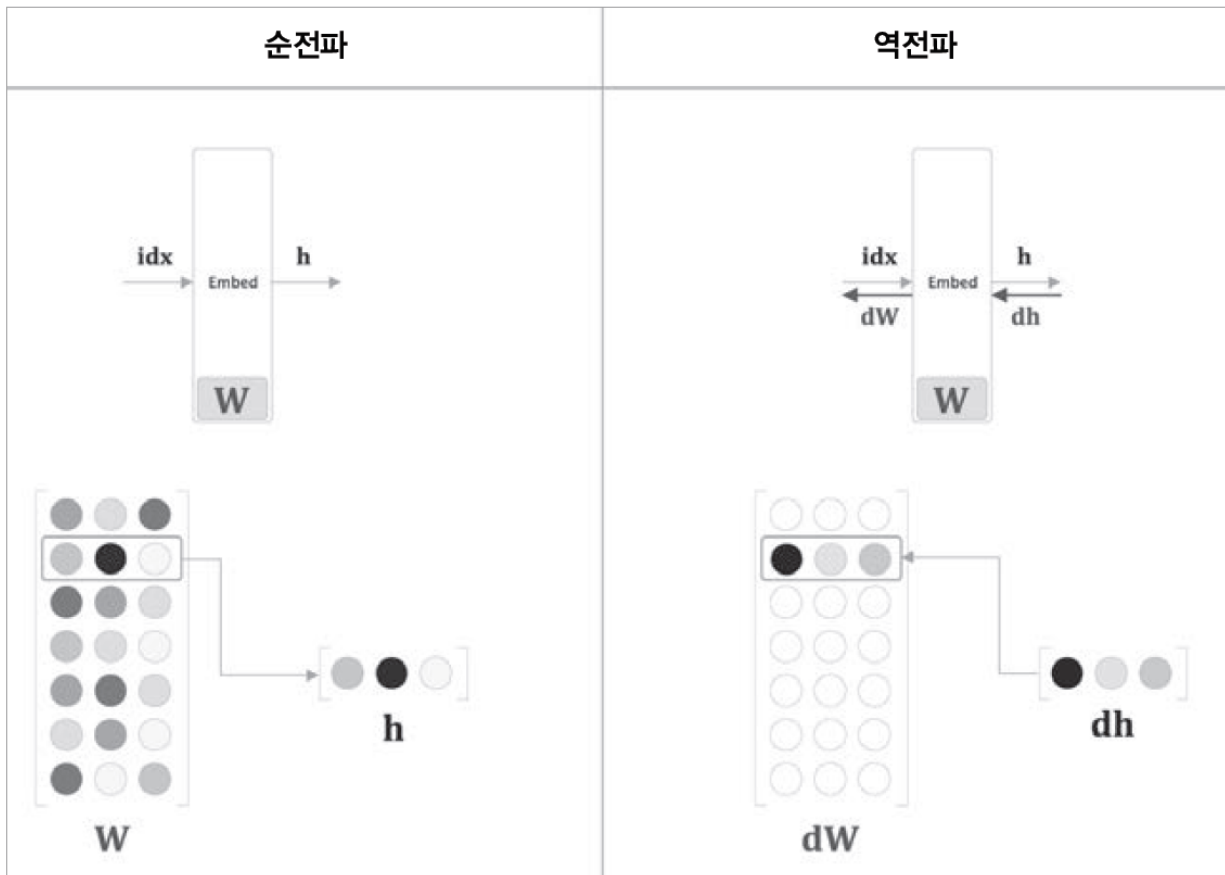
- 가중치 W로부터 여러 행을 한꺼번에 추출시
 - 원하는 행번호들을 배열에 명시하면 됨
 - 인덱스 4개 (1, 0, 3, 0)을 한번에 추출함
 - 인수에 배열을 사용하면 여러 행도
한꺼번에 추출할 수 있음

```
>>> idx = np.array([1, 0, 3, 0])
>>> W[idx]
array([[ 3,  4,  5],
       [ 0,  1,  2],
       [ 9, 10, 11],
       [ 0,  1,  2]])
```

4.1.2 Embedding 계층 구현 (1)

- Embedding 계층의 forward와 backward 처리

그림 4-4 Embedding 계층의 forward와 backward 처리(Embedding 계층은 Embed로 표기)



4.1.2 Embedding 계층 구현 (2)

■ Embedding 계층의 forward() 메소드 구현

- 인스턴스 변수 params와 grads를 사용
- idx는 추출하는 행의 인덱스를 배열로 저장
- 가중치 W의 특정행을 추출 할 뿐임
- 가중치의 특정 행 뉴런만을 다음층으로 흘려보냄

```
class Embedding:
    def __init__(self, W):
        self.params = [W]
        self.grads = [np.zeros_like(W)]
        self.idx = None

    def forward(self, idx):
        W, = self.params
        self.idx = idx
        out = W[idx]
        return out
```

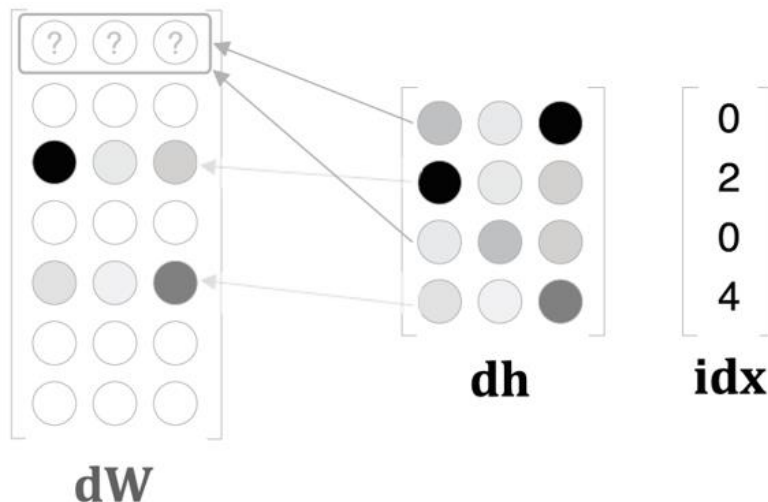
■ backward() 메소드 구현

- 앞 층으로부터 전해진 기울기를 다음 층으로 그대로 흘려줌
- 가중치 기울기 dW를 꺼낸 다음 dW의 원소를 0으로 덮어쓰
- 기울기를 가중치 기울기 dW의 특정 행(idx 번째 행)에 설정함

```
def backward(self, dout):
    dW, = self.grads
    dW[...] = 0
    dW[self.idx] = dout #실은 나쁜 예
    return None
```


4.1.2 Embedding 계층 구현 (3)

- backward() 구현 문제점
 - idx의 원소가 중복될 때 발생함
 - idx 배열의 원소중 값(행 번호)이 같은 원소가 있다면, dh를 해당 행에 할당할 때 문제가 생김
→ 먼저 쓴 값을 덮어 씌움.



- backward() 중복 문제 해결
 - '할당'이 아닌 '더하기'를 해야 함
 - dh 의 각 행의 값을 dW 의 해당 행에 더해줌

```
def backward(self, dout):
    dW, = self.grads
    dW[...] = 0
    np.add.at(dW, self.idx, dout)
    return None
```

- word2vec 의 구현
 - 입력 층 MatMul 계층을 Embedding 계층으로 전환
 - 메모리 사용량을 줄이고 쓸데없는 계산도 생략할 수 있게 됨

4.2 word2vec 개선 (2)

- 은닉층 이후의 병목 처리
 - 행렬 W_{out} 곱과 Softmax 계층의 계산 병목을 해소
 - 네거티브 샘플링(부정적 샘플링) 기법 사용
 - Softmax 대신 네거티브 샘플링을 이용하면 어휘가 많아져도 계산량을 낮은 수준에서 일정하게 억제할 수 있음

4.2.1 은닉층 이후 계산의 문제점

■ 은닉층 이후의 처리에서 계산이 오래 걸리는 곳

■ 은닉층의 뉴런과 가중치 행렬 W_{out} 의 곱

- 거대한 행렬을 곱하는 문제
- 은닉층의 벡터 크기 100
- 100 X 100만
- 역전파 때도 같은 계산을 수행
- 행렬 곱을 가볍게 만들어야 함

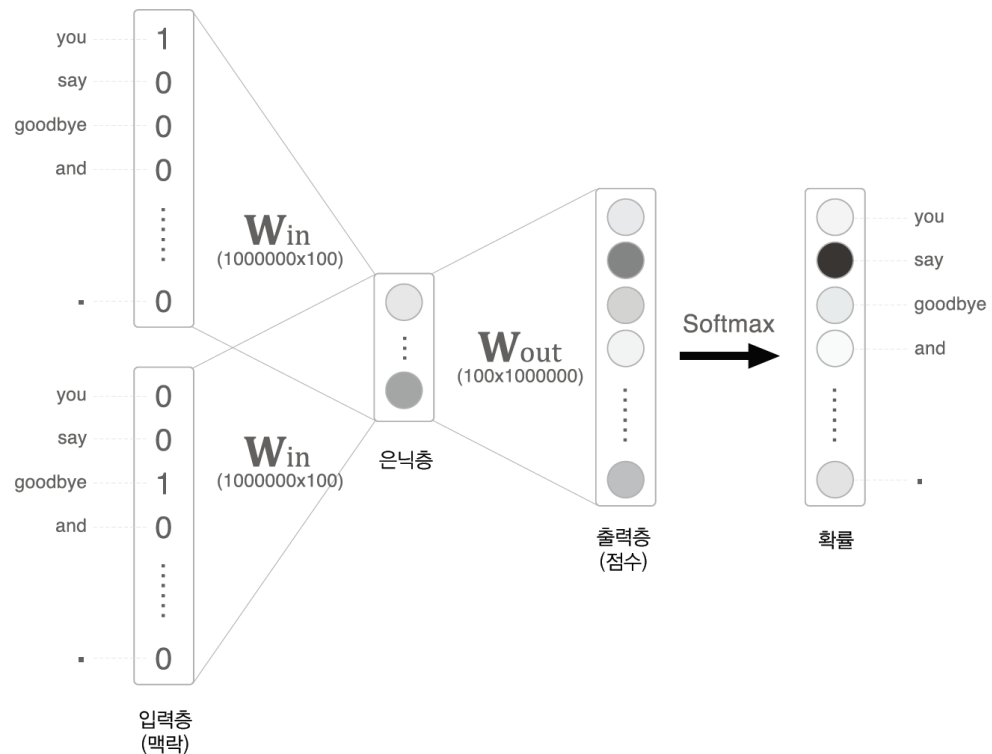
■ Softmax 계층의 계산

- 어휘가 많아지면 Softmax의 계산량도 증가

$$y_k = \frac{\exp(s_k)}{\sum_{i=1}^{1000000} \exp(s_i)}$$

- exp 계산을 100만번 수행해야 함
- Softmax를 대신할 가벼운 계산 필요

그림 4-6 어휘가 100만 개일 때를 가정한 word2vec: “you”와 “goodbye”가 맥락이고 “say”가 타깃(예측해야 할 단어)



4.2.2 다중 분류에서 이진 분류로

- 네거티브 샘플링 기법
 - 핵심 아이디어는 이진 분류(Binary Classification)
 - 다중 분류(Multi-Classification)를 이진 분류로 근사하는 것
- 다중 분류 문제를 이진 분류 방식으로 해결
 - 다중 분류 질문
 - 맥락이 주어졌을 때 정답이 되는 단어를 높은 확률로 추측하도록 학습
 - 신경망 맥락이 'you'와 'goodbye' 일때 타깃 단어는 무엇입니까?
 - 이진 분류 질문
 - 'Yes/No' 로 답할 수 있는 질문을 생각해내야 함
 - 신경망 맥락이 'you'와 'goodbye' 일때, 타깃 단어는 'say' 입니까?
 - 이진 분류 방식으로 변경하면 출력층에는 뉴런을 하나만 준비하면 됨
 - 출력층의 이 뉴런이 'say'의 점수를 출력함

4.2.2 다중 분류에서 이진 분류로 (1)

■ 타깃 단어만의 점수를 구하는 신경망

그림 4-7 타깃 단어만의 점수를 구하는 신경망

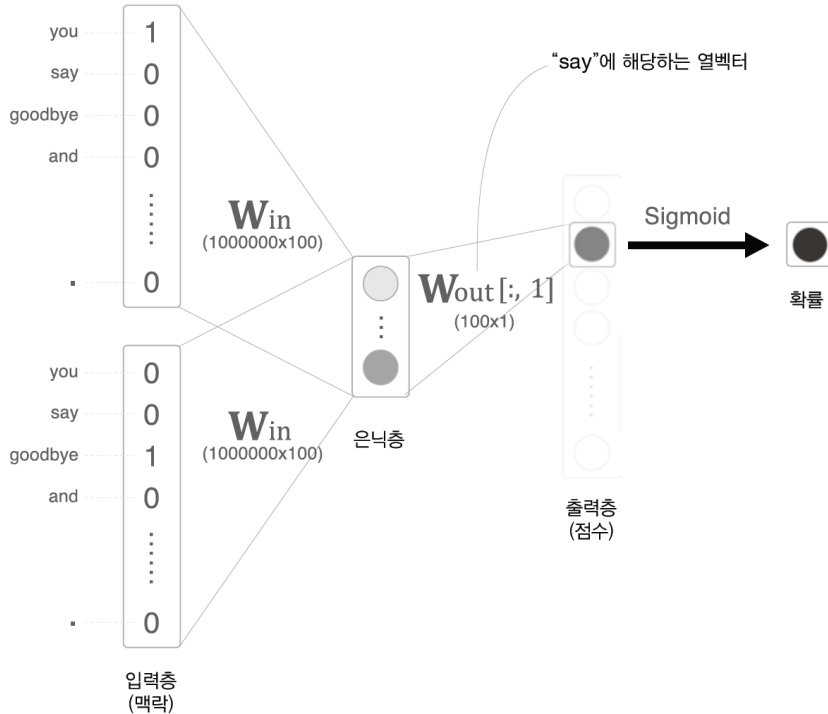
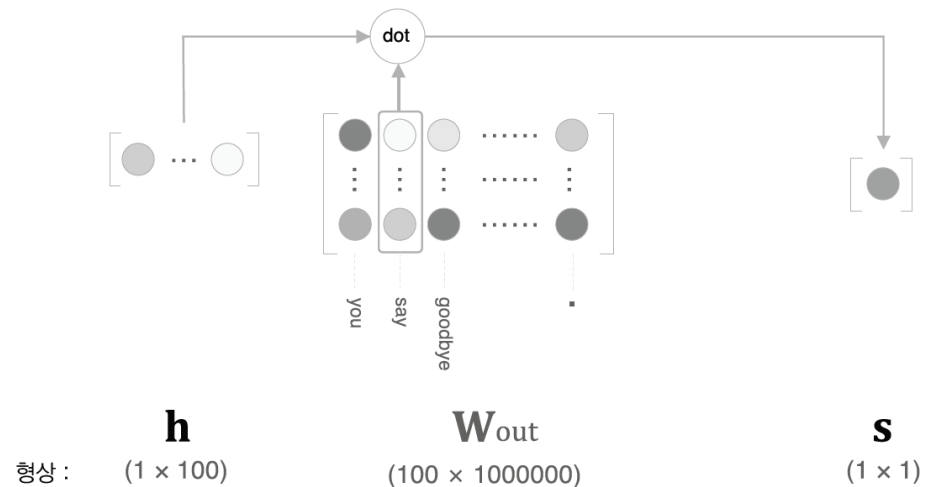


그림 4-8 "say"에 해당하는 열벡터와 은닉층 뉴런의 내적을 계산한다('dot' 노드가 내적을 계산함).



- 출력층의 뉴런은 하나임
- 은닉층과 출력층의 가중치 행렬의 내적은 'say'에 해당하는 열(단어 벡터)만 추출
- 추출된 벡터와 은닉층 뉴런과의 내적을 계산 -> 구한 값이 최종 점수

4.2.3 시그모이드 함수와 교차 엔트로피 오차

■ 이진 분류 신경망 조합

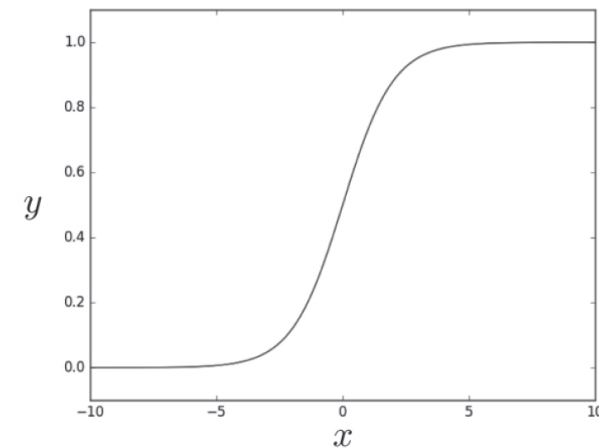
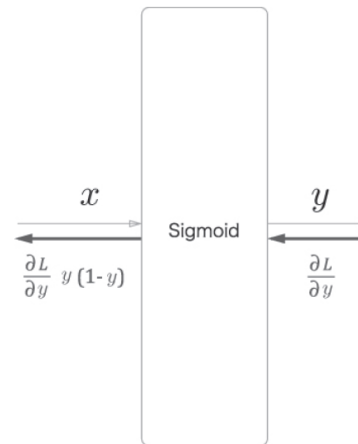
- 점수에서 시그모이드 함수를 적용해 확률로 변환
- 손실을 구할 때는 손실 함수로 '교차 엔트로피 오차'를 사용

■ 시그모이드 함수

$$y = \frac{1}{1 + \exp(-x)}$$

- 입력 값(x)은 0에서 1사이의 실수로 변환됨
- 시그모이드 함수의 출력(y)을 확률로 해석할 수 있음

그림 4-9 Sigmoid 계층(왼쪽)과 시그모이드 함수의 그래프(오른쪽)



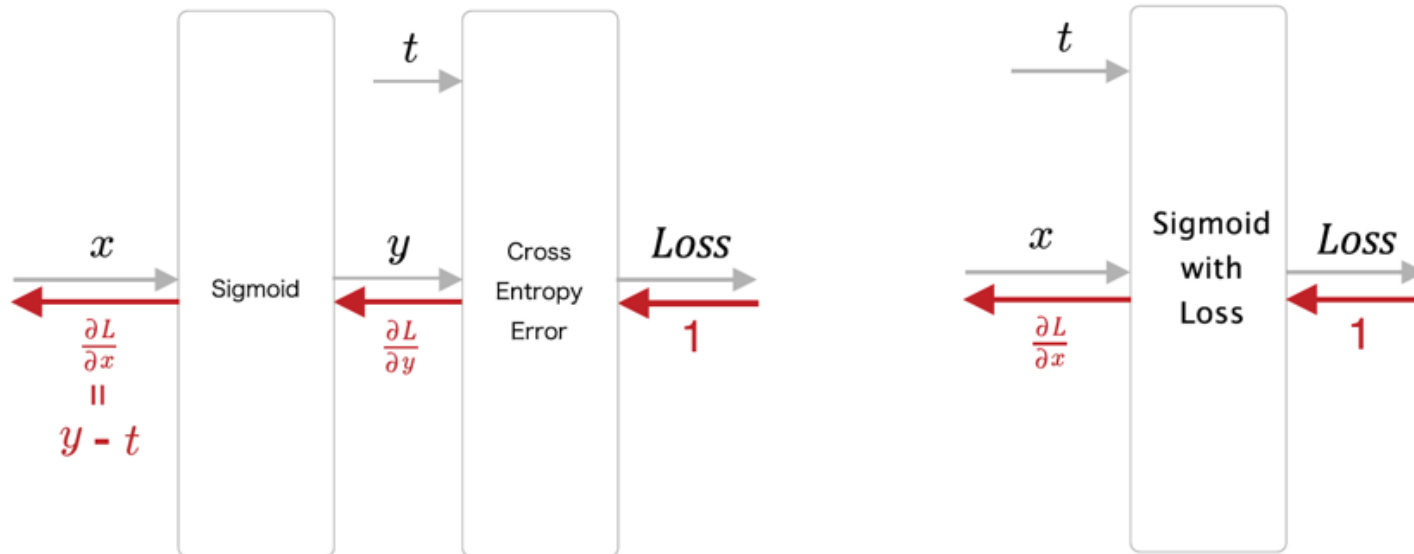
- 확률 y 로부터 교차 엔트로피 오차를 이용하여 손실을 구함.

$$L = -(t \log y + (1 - t) \log(1 - y))$$

- y 는 시그모이드 함수의 출력, t 는 정답 레이블임. t 가 1이면 정답이 'Yes'임.

4.2.3 시그모이드 함수와 교차 엔트로피 오차(1)

- Sigmoid 계층과 Cross Entropy Error 계층의 계산 그래프

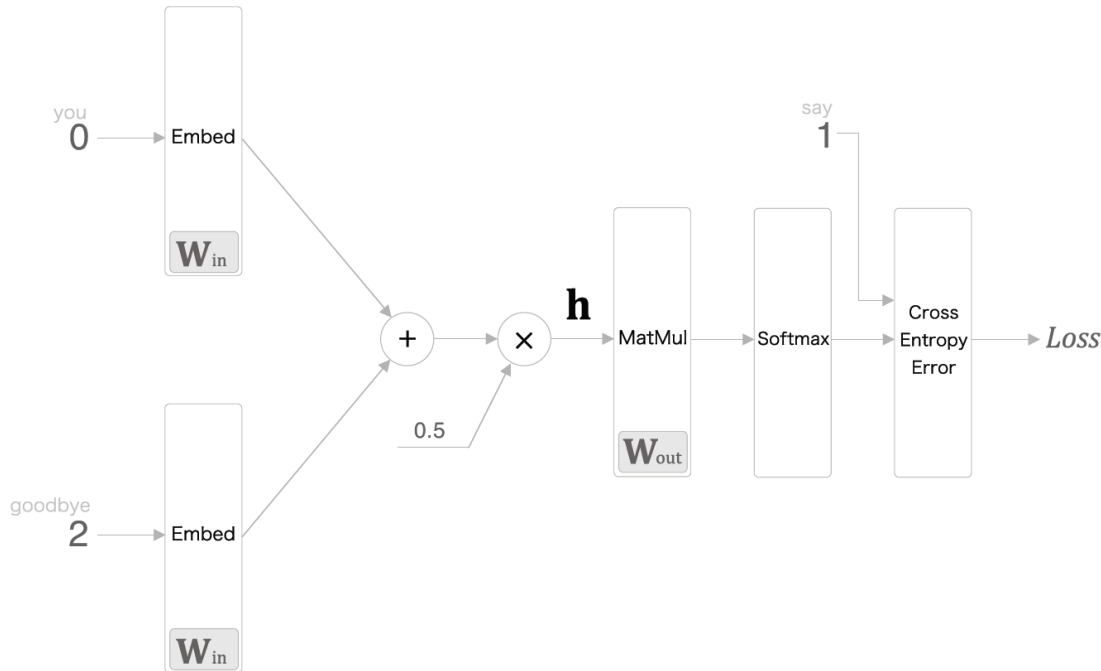


- 역전파의 $y - t$ 값은 두 값의 차이임 (y 는 신경망이 출력한 확률이고, t 는 정답 레이블임)
- 오차가 앞 계층으로 흘러감으로 오차가 크면 크게 학습하고, 오차가 작으면 작게 학습하게 됨

4.2.4 다중 분류에서 이진 분류로(구현)

- 다중 분류에 이용되는 신경망을 계층과 연산 중심으로 그린 그림

그림 4-11 다중 분류를 수행하는 CBOW 모델의 전체 그림(Embedding 계층은 Embed로 표기)

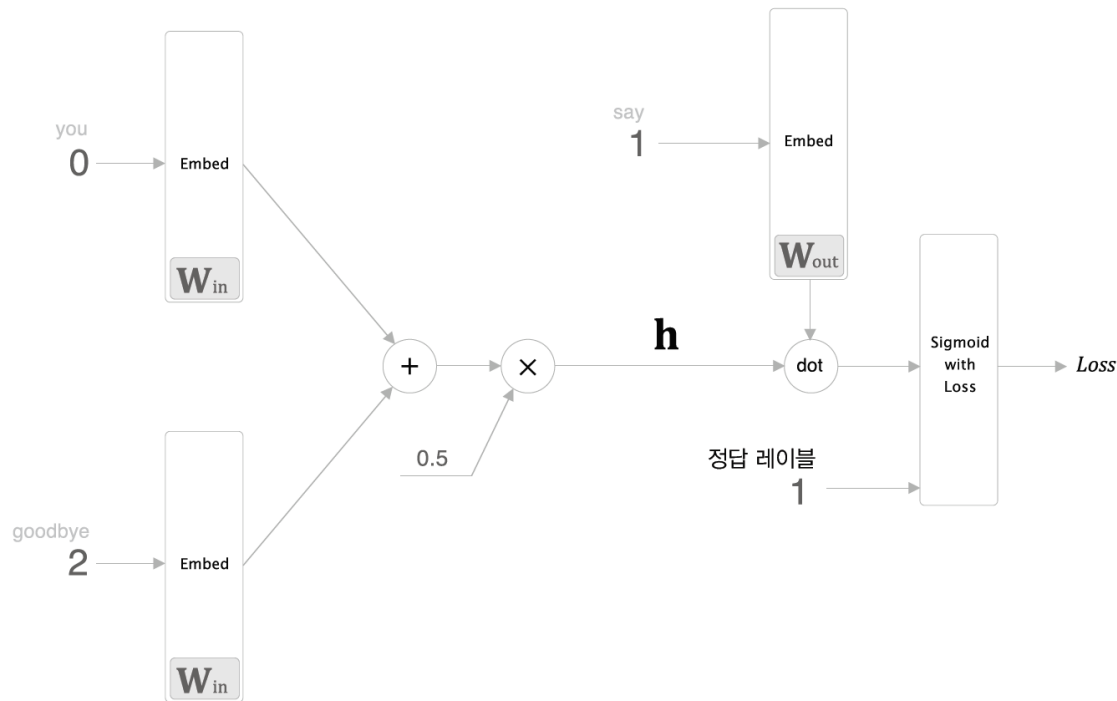


- 출력층이 어휘 수만큼의 뉴런을 준비하고 이 뉴런들이 출력한 값을 Softmax 계층에 통과시킴
- 입력층에서는 각각에 대응하는 단어 ID의 분산 표현을 추출하기 위해 Embedding 계층을 사용함

4.2.4 다중 분류에서 이진 분류로(구현) (1)

■ 다중 분류 신경망을 이진 분류 신경망으로 변환

그림 4-12 이진 분류를 수행하는 word2vec(CBOW 모델)의 전체 그림

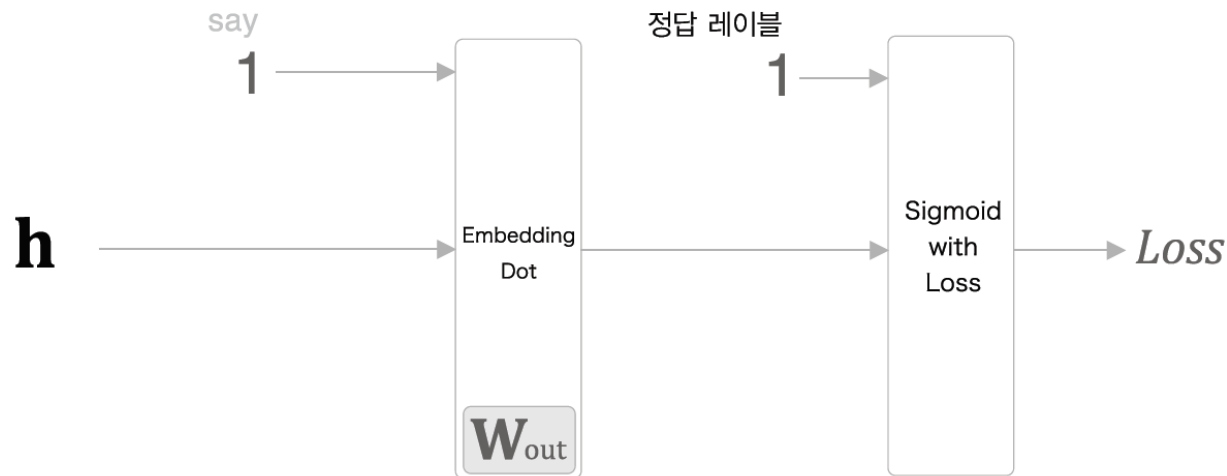


- 은닉층 뉴런 h 와 출력층의 가중치 W_{out} 에서 단어 'say'에 해당하는 단어 벡터와의 내적을 계산함
- 그 출력을 Sigmoid with Loss 계층에 입력해 최종 손실을 얻음

4.2.4 다중 분류에서 이진 분류로(구현) (2)

- Embedding Dot 계층 도입
 - Embedding 계층과 'dot 연산(내적)'의 처리를 합친 계층

그림 4-13 [그림 4-12]의 은닉층 이후 처리(Embedding Dot 계층을 사용하여 Embedding 계층과 내적 계산을 한 번에 수행)



- 은닉층 뉴런 h 는 Embedding Dot 계층을 거쳐 Sigmoid with Loss 계층을 통과
- Embedding Dot 계층을 사용하면 은닉층 이후의 처리가 간단해짐

4.2.4 다중 분류에서 이진 분류로(구현) (3)

- Embedding Dot 계층 구현
 - embed는 Embedding 계층
 - cache는 순전파 시의 계산 결과를 잠시 유지하기 위한 변수
 - idx는 단어 ID의 배열, 미니배치 처리를 가정
- 내적 계산

그림 4-14 Embedding Dot 계층의 각 변수의 구체적인 값

```
embed = Embedding(W)
target_W = embed.forward(idx)
out = np.sum(target_W * h, axis=1)
```

W	idx	target_W	h	target_W * h	out
[[0 1 2]	[0 3 1]	[[0 1 2]	[[0 1 2]	[[0 1 4]	[5 122 86]
[3 4 5]		[9 10 11]	[3 4 5]	[27 40 55]	
[6 7 8]		[3 4 5]	[6 7 8]	[18 28 40]	
[9 10 11]					
[12 13 14]					
[15 16 17]					
[18 19 20]					

```
class EmbeddingDot:
    def __init__(self, W):
        self.embed = Embedding(W)
        self.params = self.embed.params
        self.grads = self.embed.grads
        self.cache = None

    def forward(self, h, idx):
        target_W = self.embed.forward(idx)
        out = np.sum(target_W * h, axis=1)

        self.cache = (h, target_W)
        return out

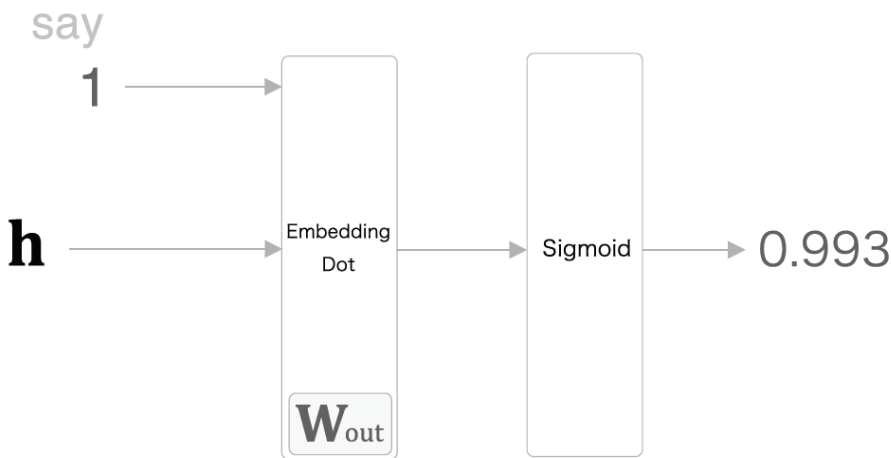
    def backward(self, dout):
        h, target_W = self.cache
        dout = dout.reshape(dout.shape[0], 1)

        dtarget_W = dout * h
        self.embed.backward(dtarget_W)
        dh = dout * target_W
        return dh
```

4.2.5 네거티브 샘플링

- 이진 분류에서 부정적인 예를 입력시 처리
 - 긍정적인 예에 대해서만 학습
 - 좋은 가중치가 준비되어 있다면 Sigmoid 계층의 출력(확률)은 1에 가까움

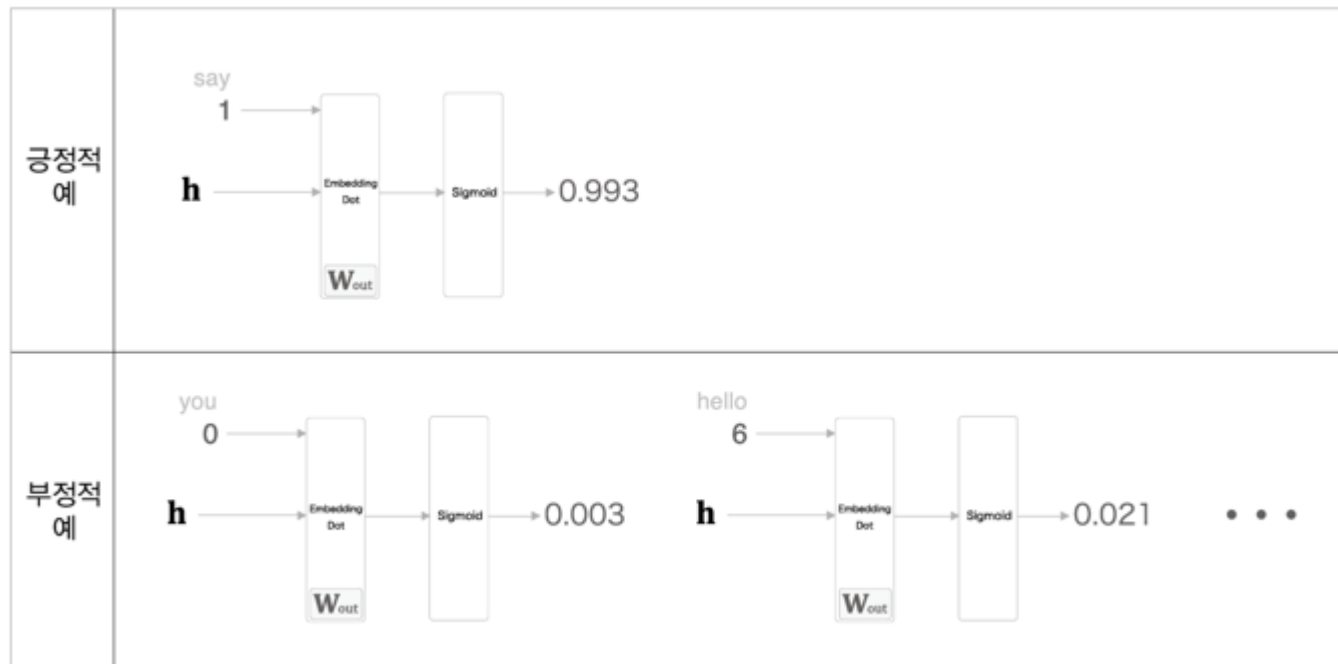
그림 4-15 CBOW 모델의 은닉층 이후의 처리 예: 맥락은 “you”와 “goodbye”이고, 타깃이 “say”일 확률은 0.993(99.3%)이다.



- 현재의 신경망에서는 긍정적인 예인 'say'에 대해서만 학습
- 부정적인 예 ('say' 이외의 단어)에 대해서는 어떠한 지식도 획득하지 못함

4.2.5 네거티브 샘플링 (1)

- 정답과 오답 각각에 대해 분류
 - 긍정적 예('say')에 대해서는 Sigmoid 계층의 출력을 1에 가깝게 만듦
 - 부정적 예('say' 이외의 단어)는 Sigmoid 계층의 출력을 0에 가깝게 만듦



- 맥락이 'you'와 'goodbye' 일 때, 타깃이 'hello'일 확률은 낮은 값이어야 함.
- 이런 결과를 만들어 주는 가중치가 필요함

4.2.5 네거티브 샘플링 (2)

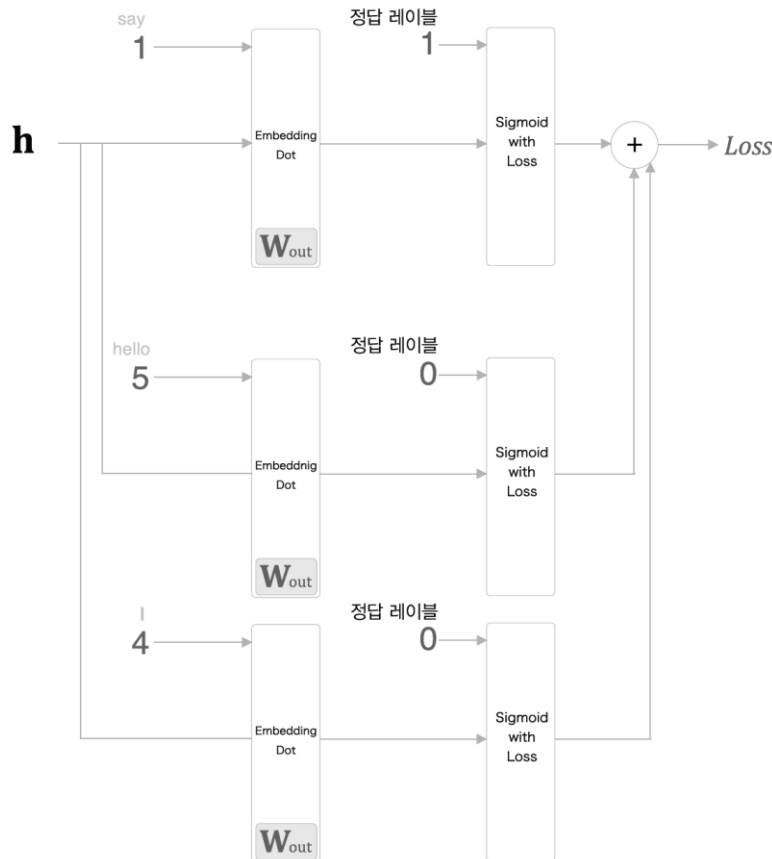
- 모든 부정적 예를 대상으로 하여 이진 분류를 학습
 - 모든 부정적 예를 대상으로 하는 방법은 어휘 수가 늘어나면 감당 할 수 없음
 - 근사적인 해법으로 부정적 예를 몇 개 선택함
- 네거티브 샘플링 기법
 - 적은 수의 부정적 예를 샘플링 하여 사용
 - 긍정적 예를 타깃으로 한 경우의 손실을 구함
 - 부정적 예를 몇 개 샘플링하여, 부정적 예에 대해서도 손실을 구함
 - 긍정적 예와 샘플링된 부정적 예의 손실을 더한 값을 최종 손실로 함
 - 구체적인 예
 - 긍정적 예의 타깃은 'say'를 다룸
 - 부정적 예의 타깃은 2개 ('hello'와 'I') 샘플링을 했다고 가정
 - CBOW 모델의 은닉층 이후만 주목

4.2.5 네거티브 샘플링 (3)

■ 네거티브 샘플링의 계산 그래프

- 긍정적 예('say')는 Sigmoid with Loss 계층에서 정답 레이블로 '1'을 입력
- 부정적 예('hello'와 'I')는 Sigmoid with Loss 계층에 정답 레이블로 '0'을 입력

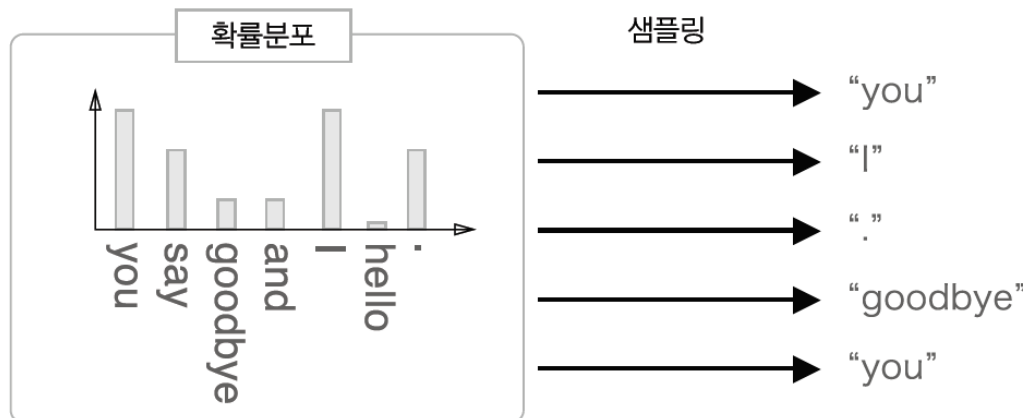
그림 4-17 네거티브 샘플링의 예(은닉층 이후의 처리에 주목하여 그린 계산 그래프)



4.2.6 네거티브 샘플링의 샘플링 기법

- 부정적인 예를 샘플링하는 방법
 - 말뭉치의 통계 데이터를 기초로 샘플링하는 방법
 - 말뭉치에서 자주 등장하는 단어를 많이 추출하고 드물게 등장하는 단어를 적게 추출함 (단어 빈도를 기준으로 샘플링)
 - 말뭉치에서 각 단어의 출현 횟수를 구해 확률분포로 나타냄
 - 확률분포대로 단어를 샘플링하면 됨
 - 확률분포대로 샘플링하므로 말뭉치에서 자주 등장하는 단어는 선택될 가능성이 높은 반면, '희소한 단어'는 선택되기 어려움

그림 4-18 확률분포에 따라 샘플링을 여러 번 수행한다.



4.2.6 네거티브 샘플링의 샘플링 기법 (1)

■ 확률분포에 따라 샘플링하는 코드

■ 무작위 샘플링 코드

```
>>> import numpy as np
>>> np.random.choice(10)
9
>>> np.random.choice(10)
7
>>> words = ['you', 'say', 'goodbye', 'I', 'hello', '.']
>>> np.random.choice(words)
'you'
>>> np.random.choice(words, size=5)
array(['say', 'goodbye', 'hello', 'you', 'goodbye'], dtype='<U7')
>>> np.random.choice(words, size=5, replace=False)
array(['you', 'I', 'hello', 'goodbye', 'say'], dtype='<U7')
```

■ 확률분포대로 샘플링 코드 (인수 p에 확률분포를 담은 리스트 지정)

```
>>> p = [0.5, 0.1, 0.05, 0.2, 0.05, 0.1]
>>> np.random.choice(words, p=p)
'you'
```

■ 확률이 낮은 단어의 확률을 높이는 방법

■ 출현 확률이 낮은 단어를 버리지 않기 위해 (0.75 이외의 다른 값 설정 가능)

$$P'(w_i) = \frac{P(w_i)^{0.75}}{\sum_j^n P(w_j)^{0.75}}$$

```
>>> p = [0.7, 0.29, 0.01]
>>> new_p = np.power(p, 0.75)
>>> new_p /= np.sum(new_p)
>>> print(new_p)
[0.64196878 0.33150408 0.02652714]
```

4.2.6 네거티브 샘플링의 샘플링 기법 (2)

- 부정적 예를 샘플링 처리 담당하는 UnigramSampler 클래스
 - 초기화 시 3개의 인수를 받음
 - 단어 ID 목록인 corpus, 확률분포에 제공할 값(0.75), 부정적 예 샘플링 sample_size
 - get_negative_sample(target) 메서드
 - 부정적 예를 골라줌 (target 인수로 지정한 단어를 긍정적 예로 해석하고 그외 단어를 선택)

```
corpus = np.array([0, 1, 2, 3, 4, 1, 2, 3])  
power = 0.75  
sample_size = 2
```

```
sampler = UnigramSampler(corpus, power, sample_size)  
target = np.array([1, 3, 0])  
negative_sample = sampler.get_negative_sample(target)  
print(negative_sample)
```

```
[[0 4]  
 [4 2]  
 [2 3]]
```

- 긍정적 예로 [1, 3, 0] 이라는 3개의 데이터를 미니배치로 다룸
- 이 각각의 데이터에 대해서 부정적 예를 2개씩 샘플링함

4.2.7 네거티브 샘플링 구현

- 네거티브 샘플링 NegativeSamplingLoss 클래스 구현
 - loss_layers 와 embed_dot_layers 리스트에 sample_size + 1 개의 계층 생성
 - 부정적인 예를 다루는 계층이 sample_size 개임
 - 0번째 계층이 긍정적인 예를 다루는 계층임
 - 이 계층에서 사용하는 매개변수와 기울기를 각각 배열로 저장함

```
class NegativeSamplingLoss:
    def __init__(self, W, corpus, power=0.75, sample_size=5):
        self.sample_size = sample_size
        self.sampler = UnigramSampler(corpus, power, sample_size)
        self.loss_layers = [SigmoidWithLoss() for _ in range(sample_size + 1)]
        self.embed_dot_layers = [EmbeddingDot(W) for _ in range(sample_size + 1)]

        self.params, self.grads = [], []
        for layer in self.embed_dot_layers:
            self.params += layer.params
            self.grads += layer.grads
```

4.2.7 네거티브 샘플링 구현(1)

■ NegativeSamplingLoss 클래스 순전파와 역전파 구현

- 인수 - 은닉층 뉴런 h, 긍정적 예의 타깃을 뜻하는 target
- 긍정적인 예와 부정적 예 각각의 데이터에 대해서 순전파를 수행해 그 손실들을 더함
- correct_label은 1 이고, negative_label 은 0임

```
def forward(self, h, target):  
    batch_size = target.shape[0]  
    negative_sample = self.sampler.get_negative_sample(target)  
  
    score = self.embed_dot_layers[0].forward(h, target)  
    correct_label = np.ones(batch_size, dtype=np.int32)  
    loss = self.loss_layers[0].forward(score, correct_label)  
  
    negative_label = np.zeros(batch_size, dtype=np.int32)  
    for i in range(self.sample_size):  
        negative_target = negative_sample[:, i]  
        score = self.embed_dot_layers[1 + i].forward(h, negative_target)  
        loss += self.loss_layers[1 + i].forward(score, negative_label)  
  
    return loss
```

- 역전파 - 여러 개의 기울기 값을 더해줌

```
def backward(self, dout=1):  
    dh = 0  
    for l0, l1 in zip(self.loss_layers, self.embed_dot_layers):  
        dscore = l0.backward(dout)  
        dh += l1.backward(dscore)  
    return dh
```

4.3 개선판 word2vec 학습

- Embedding 계층을 설명
- 네거티브 샘플링 기법 설명
- 두 가지를 실제로 구현
- 개선점을 신경망 구현에 적용
- PTB 데이터셋을 사용해 학습하고 더 실용적인 단어의 분산 표현을 얻음

4.3.1 CBOW 모델 구현

- 3장의 SimpleCBOW 클래스를 개선
 - 개선점은 Embedding 계층과 Negative Sampling Loss 계층을 적용
 - 가중치 초기화 후 계층을 생성
 - Embedding 계층을 $2 * \text{window_size}$ 개 작성해 인스턴스 변수인 `in_layer`에 배열로 보관
 - Negative Sampling Loss 계층을 생성
 - 단어의 분산 표현에 접근 할 수 있도록 `word_vecs`에 가중치 W_{in} 을 할당함

```
class CBOW:
    def __init__(self, vocab_size, hidden_size, window_size, corpus):
        V, H = vocab_size, hidden_size
        W_in = 0.01 * np.random.randn(V, H).astype('f')
        W_out = 0.01 * np.random.randn(V, H).astype('f')

        self.in_layers = []
        for i in range(2 * window_size):
            layer = Embedding(W_in) # Embedding 계층 사용
            self.in_layers.append(layer)
        self.ns_loss = NegativeSamplingLoss(W_out, corpus, power=0.75, sample_size=5)

        layers = self.in_layers + [self.ns_loss]
        self.params, self.grads = [], []
        for layer in layers:
            self.params += layer.params
            self.grads += layer.grads

        # 인스턴스 변수에 단어의 분산 표현을 저장한다.
        self.word_vecs = W_in
```

4.3.1 CBOW 모델 구현 (1)

- SimpleCBOW 클래스 순전파와 역전파
 - 각 계층의 순전파(혹은 역전파)를 적절한 순서로 호출
 - forwar(context, target) 메서드가 인수로 받는 맥락과 타깃이 단어 ID 임

그림 4-19 맥락과 타깃을 단어 ID로 나타낸 예(맥락의 윈도우 크기는 1)



```
def forward(self, contexts, target):
    h = 0
    for i, layer in enumerate(self.in_layers):
        h += layer.forward(contexts[:, i])
    h *= 1 / len(self.in_layers)
    loss = self.ns_loss.forward(h, target)
    return loss
```

```
def backward(self, dout=1):
    dout = self.ns_loss.backward(dout)
    dout *= 1 / len(self.in_layers)
    for layer in self.in_layers:
        layer.backward(dout)
    return None
```

4.3.2 CBOW 모델 학습 코드

- CBOW 모델의 학습을 구현 – 신경망 학습을 수행
 - 윈도우 크기를 5, 은닉층 뉴런 수를 100개 설정
 - 파이썬 코드의 객체를 파일로 저장하는 피클을 이용하여 파일 저장

```
import pickle
from common.trainer import Trainer
from common.optimizer import Adam
from cbow import CBOW
from skip_gram import SkipGram
from common.util import create_contexts_target, to_cpu, to_gpu
from dataset import ptb
```

하이퍼파라미터 설정

```
window_size = 5
hidden_size = 100
batch_size = 100
max_epoch = 10
```

데이터 읽기

```
corpus, word_to_id, id_to_word = ptb.load_data('train')
vocab_size = len(word_to_id)
```

```
contexts, target = create_contexts_target(corpus, window_size)
```

```
if config.GPU:
    contexts, target = to_gpu(contexts), to_gpu(target)
```

```
model = CBOW(vocab_size, hidden_size, window_size, corpus)
optimizer = Adam()
trainer = Trainer(model, optimizer)
```

학습 시작

```
trainer.fit(contexts, target, max_epoch, batch_size)
trainer.plot()
```

나중에 사용할 수 있도록 필요한 데이터 저장

```
word_vecs = model.word_vecs
if config.GPU:
    word_vecs = to_cpu(word_vecs)
params = {}
params['word_vecs'] = word_vecs.astype(np.float16)
params['word_to_id'] = word_to_id
params['id_to_word'] = id_to_word
pkl_file = 'cbow_params.pkl' # or 'skipgram_params.pkl'
with open(pkl_file, 'wb') as f:
    pickle.dump(params, f, -1)
```


4.3.3 CBOW 모델 평가

■ 학습한 단어의 분산 표현을 평가

- 결과를 보면 CBOW 모델로 획득된 단어의 분산 표현이 잘 구성되어 보임
- word2vec으로 얻은 단어의 분산 표현은 비슷한 단어를 가까이 모음을 볼수 있음

```
import sys
sys.path.append('.')
from common.util import most_similar, analogy
import pickle
```

```
pkl_file = 'cbow_params.pkl'
# pkl_file = 'skipgram_params.pkl'
```

```
with open(pkl_file, 'rb') as f:
    params = pickle.load(f)
    word_vecs = params['word_vecs']
    word_to_id = params['word_to_id']
    id_to_word = params['id_to_word']
```

```
# 가장 비슷한(most similar) 단어 뽑기
```

```
querys = ['you', 'year', 'car', 'toyota']
```

```
for query in querys:
```

```
    most_similar(query, word_to_id, id_to_word, word_vecs, top=5)
```

```
[query] you
we: 0.6103515625
someone: 0.59130859375
i: 0.55419921875
something: 0.48974609375
anyone: 0.47314453125
```

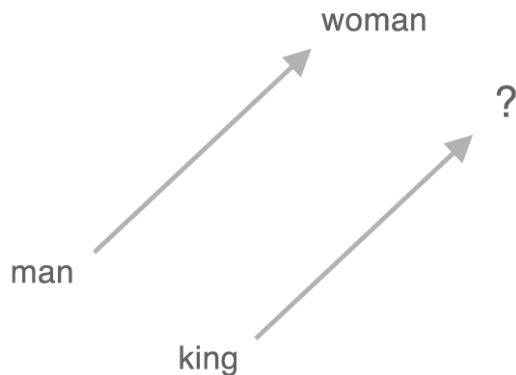
```
[query] year
month: 0.71875
week: 0.65234375
spring: 0.62744140625
summer: 0.6259765625
decade: 0.603515625
```

```
[query] car
luxury: 0.497314453125
arabia: 0.47802734375
auto: 0.47119140625
disk-drive: 0.450927734375
travel: 0.4091796875
```

4.3.3 CBOW 모델 평가 (1)

- 단어의 분산 표현을 이용한 유추 문제 해결
 - 단어의 분산 표현을 사용하면 유추 문제를 벡터의 덧셈과 뺄셈으로 풀 수 있음
 - 단어의 단순한 의미뿐 아니라 문법적인 패턴도 파악할 수 있음
 - 단어 벡터 공간에서 "man -> woman" 벡터일 때 "king -> ?" 유추 문제

그림 4-20 "man : woman = king : ?" 유추 문제 풀기(단어 벡터 공간에서 각 단어의 관계성)



```
# 유추(analogy) 작업
print('-'*50)
analogy('king', 'man', 'queen', word_to_id, id_to_word, word_vecs)
analogy('take', 'took', 'go', word_to_id, id_to_word, word_vecs)
analogy('car', 'cars', 'child', word_to_id, id_to_word, word_vecs)
analogy('good', 'better', 'bad', word_to_id, id_to_word, word_vecs)
```

```
[analogy] king:man = queen:?
woman: 5.16015625
veto: 4.9296875
ounce: 4.69140625
earthquake: 4.6328125
successor: 4.609375
```

```
[analogy] take:took = go:?
went: 4.55078125
points: 4.25
began: 4.09375
comes: 3.98046875
oct.: 3.90625
```

```
[analogy] car:cars = child:?
children: 5.21875
average: 4.7265625
yield: 4.20703125
cattle: 4.1875
priced: 4.1796875
```

4.4 word2vec 남은 주제

4.4.1 word2vec 을 사용한 어플리케이션의 예

■ 단어의 분산 표현의 장점

■ 전이 학습 (transfer learning)

- 자연어 처리 분야에서 단어의 분산 표현이 중요한 이유
- 한 분야에서 배운 지식을 다른 분야에도 적용하는 기법
- 자연어 문제를 풀 때 word2vec의 단어 분산 표현을 처음부터 학습하지 않음
- 큰 말뭉치로 학습을 끝난 후, 그 분산 표현을 각자의 작업에 이용함

■ 단어를 고정 길이 벡터로 변환

- 문장(단어의 흐름)도 단어의 분산 표현을 이용하여 고정 길이 벡터로 변환 할 수 있음
- 자연어를 벡터로 변환할 수 있다면 머신러닝 기법(신경망, SVM)을 적용할 수 있음

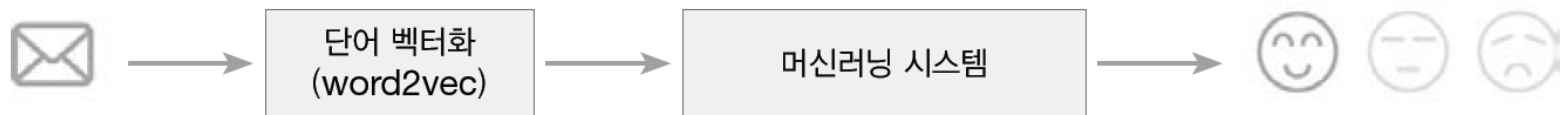
그림 4-21 단어의 분산 표현을 이용한 시스템의 처리 흐름



4.4.1 word2vec 을 사용한 어플리케이션의 예

- 단어의 분산 표현 사용법의 구체적인 예
 - 자연어를 다루는 문제는 단어의 분산 표현으로 벡터화 할 수 있음
 - 벡터화 후 머신러닝 기법으로 해결 and word2vec 전이 학습의 혜택을 누림
 - 메일을 자동으로 분류하는 시스템
- 메일을 자동으로 분류하는 시스템
 - 스마트폰 앱 사용자 1억명으로 부터 호의적인 의견과 불만의 메일이 옴
 - 메일 내용을 보고 사용자의 감정을 3단계로 분류하고자 함
 - 정확히 분류할 수 있다면 불만을 가진 사용자의 메일부터 보고 조치를 취함

그림 4-22 메일 자동 분류 시스템의 예(감정 분석)



- 시스템 개발 과정
 - 메일 데이터를 수집하고, 모은 메일들에 수동으로 레이블을 붙임
 - Word2vec 을 이용해 메일을 벡터로 변환 후 머신러닝으로 학습 수행

4.4.2 단어 벡터 평가 방법

- 단어의 분산 표현을 평가하는 방법
 - word2vec 통해 얻은 단어의 분산 표현이 좋은지 어떻게 평가할까?
 - 단어의 분산 표현을 만드는 시스템과 분류하는 시스템의 학습을 분리
 - 단어의 분산 표현을 학습하고, 그 분산 표현을 사용하여 머신러닝 시스템을 학습
 - 두 단계의 학습을 수행한 후 평가
 - 단어의 분산 표현의 우수성을 실제 어플리케이션과는 분리해 평가
 - 평가 척도는 단어의 '유사성'이나 '유추 문제'를 활용한 평가
 - 단어의 유사성 평가
 - 사람이 작성한 단어 유사도를 사용해 평가 (유사도를 0 ~ 10 사이로 점수화)
 - 사람이 부여한 점수와 word2vec에 의한 코사인 유사도 점수를 비교
 - 유추 문제를 활용한 평가
 - "king : queen = man : ?" 와 같은 유추 문제를 출제
 - 유추 문제의 정답률로 단어의 분산 표현의 우수성을 측정

4.4.2 단어 벡터 평가 방법 (1)

■ 유추 문제에 의한 단어 벡터의 평가 결과

모델	차수	말뭉치 크기	의미(semantics)	구문(syntax)	종합
CBOW	300	16억	16.1	52.6	36.1
skip_gram	300	10억	61	61	61
CBOW	300	60억	63.6	67.4	65.7
skip_gram	300	60억	73.0	66.0	69.1
CBOW	1000	60억	57.3	68.9	63.7
skip_gram	1000	60억	66.1	65.1	65.6

- 의미열 – 단어의 의미를 유추하는 유추 문제의 정답률 (king : queen = actor : ?)
- 구문열 – 단어의 형태 정보를 묻는 문제 (bad : worst = good : ?)
- 유추 문제 측정을 통해 알수 있는것
 - 단어의 의미나 문법적인 문제를 제대로 이해하고 있는지 파악
 - 좋은 평가를 받은 분산 표현이라면 자연어 처리시 좋은 결과를 기대할 수 있음
 - 평가가 높다고 어플리케이션에서 반드시 좋은 결과를 보장하지는 않음
(다루는 문제 상황에 따라 다름)



Thank You

